

VRadioSim: A GPU-Based Ray Tracing Simulator for Real-Time Wireless Propagation Visualisation in Virtual Reality

Louis Nisiotis
School of Sciences
UCLan Cyprus
LNisiotis@uclan.ac.uk

Marios Raspopoulos
School of Sciences
UCLan Cyprus
MRaspopoulos@uclan.ac.uk

Abstract—Conducting real-time wireless propagation simulation in interactive 3D environments is computationally demanding, and this challenge becomes more critical in Virtual Reality (VR), where stable and high frame rates are required for usability and comfort. This paper presents *VRadioSim*, a GPU-based wireless propagation simulator prototype that supports real-time visualisation of multipath rays and received-signal heatmaps in desktop and tethered VR modes. The prototype loads 3D scenes from GLB models and enables interactive placement of transmitter and receiver entities, visualisation of signal accumulation across scene surfaces, and point-based querying of received power values. A preliminary evaluation across varying scene complexities and ray counts indicates that the desktop version remains consistently interactive, and that tethered VR is currently most practical for lower complexity scenes. Signal sampling measurements show that received-power estimates are mostly consistent across different geometrical scene representations, suggesting stability of the current GPU-based propagation implementation. Overall, the findings demonstrate the practical feasibility of prototype for interactive desktop analysis and provide an initial basis for further optimisation and validation in immersive VR settings.

Index Terms—Real Time Ray Tracing, GPU, Virtual Reality, Radio Propagation Simulation, Digital Twins

I. INTRODUCTION

Wireless propagation simulations require complex multipath interactions, including reflection, refractions (or transmission through), diffractions and scattering. Understanding and modelling these effects is key for reliable placement and optimisation of wireless infrastructure, and it is increasingly relevant for emerging use cases such as smart buildings, industrial IoT, and Digital Twin deployments. However, propagation results are difficult to interpret when presented only as numerical outputs or 2D plots, particularly in geometrically complex spaces where signal behaviour relates directly to the structure and the material properties of the environment. Interactive 3D visualisation can improve interpretability by allowing users to inspect ray paths and coverage distributions directly within a spatial context. This becomes more compelling in Virtual Reality (VR), where users can navigate a scene at human scale and explore signal propagation behaviour. Yet, real-time ray-based simulation remains challenging because VR

requires stable and high frame rates to preserve usability. Based on our previous experimental efforts deploying a real time radio propagation simulator using Unreal Engine [1], [2] the requirements of such simulators that rely on heavy computations can introduce frame-time drops, requiring to reduce ray counts, limiting interaction depth, or disabling detailed ray visualisation, thereby constraining simulation fidelity.

This paper presents *VRadioSim*, a GPU-based wireless propagation simulator prototype designed for real-time analysis in desktop and tethered VR modes. The system loads 3D scenes from GLB models, supports interactive placement of transmitter and receiver entities, and visualises multipath ray trajectories, received-power heatmaps, and configurable horizontal and vertical grid planes. It uses a compute-shader ray-tracing pipeline that performs ray traversal and field accumulation directly on the GPU and feeds the results to the renderer, enabling interactive propagation visualisation for desktop and immersive VR inspection. This paper describes the system architecture, the current prototype implementation, and preliminary performance and measurement-consistency results to demonstrate the practicality of GPU-driven propagation visualisation in interactive desktop and VR environments.

II. BACKGROUND AND CONTEXT

A. Deterministic Wireless Propagation & Ray-Based Methods

Deterministic propagation models based on geometrical optics [3] and ray-based techniques are widely used for site-specific wireless channel characterization. In contrast to empirical models, deterministic approaches explicitly account for the interaction of electromagnetic waves with the environment geometry and material properties, enabling the estimation of multipath components, angular characteristics, and channel responses [4]. Ray tracing [5] methods approximate high-frequency wave propagation through mechanisms such as reflection, refraction and diffraction, and have been shown to provide high accuracy when detailed environmental models are available. However, the computational complexity of ray-based methods increases with scene complexity, propagation distance, and interaction depth, historically limiting their use to offline analysis and planning tools.

B. GPU-Accelerated Ray Tracing for Wireless Propagation

Recent advances in parallel computing architectures, particularly the widespread availability of programmable GPUs, have renewed interest in deterministic propagation models capable of operating at interactive or near-real-time rates. These developments [6]–[8] enable the integration of ray-based propagation within dynamic analysis workflows, motivating new simulation architectures that exploit massive parallelism whilst preserving physical accuracy, empowering the 6G community with tools that can effectively, efficiently and accurately simulate radio propagation. This capability has become increasingly important in light of recent advances in wireless communications, including the shift towards higher carrier frequencies in the millimeter-wave bands, the adoption of advanced antenna technologies such as phased arrays, the emergence of massive MIMO systems that explicitly exploit multipath propagation, and the growing deployment of Reconfigurable Intelligent Surfaces (RIS) [9]–[11].

The highly parallel nature of ray-based propagation makes it well-suited for GPU acceleration, where thousands of rays can be processed concurrently. Prior research has demonstrated substantial speed-ups by mapping ray launching, intersection testing and path evaluation to GPU kernels, often combined with spatial acceleration structures such as kD-trees or bounding volume hierarchies (BVHs) to reduce intersection complexity [12]. Several representative examples of GPU-accelerated propagation modeling can be found in the literature. In [6] for instance, a GPU-enabled kD-tree beam-tracing approach is achieving speedups of up to $66\times$ compared to conventional ray-tracing implementations within the evaluated scenarios. Similarly, [7] combines environment discretization, visibility preprocessing, and massive GPU parallelism, reporting computational gains of up to four orders of magnitude relative to a baseline ray-tracing algorithm.

Recent progress in GPU architectures has further expanded the potential for ultra-fast deterministic channel modeling. In this context, [8] investigates large-scale electromagnetic simulation environments and demonstrates significant acceleration in channel prediction by leveraging the NVIDIA® OptiX™ ray tracing framework. A notable recent development is the high-fidelity 5G simulation platform jointly developed by Ericsson and NVIDIA [13], exploiting accelerated computing through NVIDIA Omniverse® for dynamic evaluation of radio propagation and network performance, effectively illustrating the practical realization of a wireless digital twin [14]. Despite these advances however, many existing GPU-accelerated solutions target offline simulations rather than real-time emulators, and as a result, they lack the deterministic execution behavior required for interactive visualization or immersive applications, particularly when scene geometry or transmitter/receiver configurations change dynamically.

C. Digital Twins and Radio Propagation Modeling

Digital twins represent virtual counterparts of physical systems that are continuously synchronized with real-world data

to support monitoring, analysis, and decision-making. In wireless communications, digital twin concepts are increasingly applied to model radio environments, network infrastructure, and user mobility, giving rise to the notion of radio or network digital twins [15]–[17]. Within such twins, propagation modeling plays a key role by providing spatial coverage estimates, multipath characteristics, and what-if analysis under dynamic conditions. Deterministic ray tracing offers high-fidelity insight into these phenomena but must be tightly integrated with the twin’s computational and visualization layers to support interactive use. Real-time or near-real-time propagation simulation, combined with immersive visualization can enhance interpretability by allowing engineers and operators to directly explore spatial relationships between geometry, propagation paths, and network performance. This capability is particularly valuable for planning, optimization, and validation tasks within dynamic digital twin frameworks.

D. Prior Work and Motivation

Accurate modelling of wireless propagation in indoor environments remains challenging due to complex multipath effects, occlusions, and material-dependent interactions. High-fidelity and sophisticated commercial tools such as *Remcom Wireless InSite*, *Altair WinProp* and *RanPlan Wireless* can provide detailed results, however their outputs are often difficult to interpret and are generated offline, limiting their usefulness for rapid iteration during network planning, optimisation, or interactive Digital Twin applications. Furthermore, these tools are quite expensive, limiting their wide use implementation by smaller SMEs, educational and research organisations, as well as for individual engineers. Visual simulation in 3D, and especially in VR, can improve interpretability by allowing users to inspect ray paths and coverage distributions and perform “what-if” exploration by adjusting transmitter and receiver placement. However, achieving interactive performance is non-trivial, as ray-based propagation imposes substantial computational load, which is especially further amplified when attempting to use in real time interactive environments and in particular VR to support immersive visualisation.

Prior work in this area includes prototypes such as *DeepWiSim*, which supports deep-learning-driven automation for wireless simulation [18], and *See-the-Radio-Waves*, a VR demo prototype for visualising and optimising wireless systems [19]. Additional efforts have explored the visualisation of electromagnetic simulation outputs [20] and the development of large-scale wireless emulators that apply radio-wave propagation models within 3D environments [21]. Furthermore, a demonstration system developed by [22] uses GPU-based ray tracing to achieve interactive wireless signal propagation simulation in real time in a real world setting through rapid scene modeling, demonstrating the high performance impact of such computational approach.

In our earlier work, we developed a low-cost wireless signal propagation simulator prototype implemented in Unreal Engine 5, for a Digital Twin visual analytics tool supporting desktop and immersive VR visualisation and interaction [1],

[2]. The simulator was based on a *Transmitter–Obstacle–Receiver* architecture, emitting rays from a configurable transmitter, computing multi-interaction paths through scene geometry, and accumulating the resulting received field into a grid of receiver cells to generate a signal-strength heatmap. The prototype provided the ability to perform real-time ray visualisation in 3D and VR, enabling to navigate the scene and interpret multipath behaviour alongside coverage distributions. To evaluate the efficacy of this initial prototype, we conducted a thorough performance and device testing revealing practical configurations and relevant limitations for immersive interactive use [1]. Results showed that the prototype could effectively support an interactive Digital Twin, including scene exploration, and visualisation of multipath ray tracing, and coverage assessment through spatial heatmaps. It also demonstrated practical benefits for deployment in low-cost settings, where users can use the engine to develop high fidelity digital twins, reposition transmitter and receivers, and immediately inspect the impact on propagation patterns, making the simulator suitable for educational and exploratory use in PC and VR modes.

However, device testing showed that real-time ray visualisation substantially reduced frame rate during the computation phase as ray counts increased, often dropping below VR comfort thresholds, even when desktop performance remained usable. This created the need of balancing between usability and simulation accuracy and fidelity where to preserve VR comfort, the simulator needed conservative parameter settings such as lowered ray density, reduced interaction complexity, or disabling real-time ray rendering, which limits diagnostic detail and the achievable simulation resolution. In addition, as a result of the CPU based calculations and the engine’s overhead of operation, the system’s scalability and reliability were sensitive to hardware constraints, and the modelling accuracy was not yet validated against commercial tools or in-situ measurements, as it required further development through a more controlled performance-oriented implementation. These limitations motivate the present work for prototyping a GPU-based propagation prototype simulator that moves the dominant workload of ray generation and traversal, interaction updates, and field accumulation directly onto the GPU using compute shaders. The aim is to achieve higher ray counts, richer multipath behaviour and stable performance.

III. PROTOTYPE DEVELOPMENT

The *VRadioSim* prototype is implemented in C++ on Direct3D 12 with OpenXR for VR rendering, and provides several features for interactive propagation analysis such as loading a GLB scene, position the transmitter and receiver in real time, configure simulation parameters (e.g. carrier frequency, ray counts and bounces) and inspect results through ray-path visualisation, surface heatmaps, and configurable grid planes. The desktop application targets Windows OS and the VR rendering mode is established through tethered headset configuration using Meta Horizon Link. The simulator follows a GPU based architecture comprising i) a CPU control plane

for scene loading and user interaction, and ii) a GPU data plane that performs ray generation, ray–triangle intersection, multi-bounce propagation, and signal accumulation. Scene geometry is uploaded as triangle buffers on the GPU. A compute-shader ray tracer launches N rays from the transmitter (T_x), advances each ray for up to B interactions, writes i) ray segments for visual inspection and ii) accumulated field contributions into heatmap/receiver buffers. A subsequent GPU pass converts the accumulated field into received power and produces heatmap textures and grid-based field values for visualisation.

1) *Field Accumulation and Power Computation:* In *VRadioSim*, wireless propagation is modeled using a ray-based approximation derived from geometrical optics. As illustrated in Fig. 1a, rays are launched from the transmitter and interact with scene geometry, producing direct and multi-bounce paths (e.g. first and second-order reflections), whilst other rays are not captured due to occlusion or direction. As rays travel, their electric field contribution is reduced due to distance-dependent attenuation (path loss) and interaction losses. In the current prototype, propagation is primarily modelled through direct paths and specular reflections, with repeated interactions producing visible multipath behaviour. The respective electric field of each component is recalculated by calculating and accounting for the respective 3D reflection and transmission coefficients. Another mechanism which however is currently not supported is diffraction which is theoretically considered at the facet edges (this is part of future work and improvements). Rather than evaluating rays only at discrete points (specific user-defined receiver locations), contributions from many rays and paths are accumulated spatially and their respective complex electric field is vectorially added as they traverse the scene and deposited into surface, receiver, and grid-plane accumulation buffers. The accumulated complex electric field is then converted into received power and visualised as continuous heatmaps over scene surfaces and as sampled values across grid planes for an intuitive representation of coverage and multipath effects.

2) *GPU based computation:* The implementation is designed to keep computation and visualisation on the GPU within each frame to support real-time rendering. The propagation step produces updated ray paths and heatmap values directly on the GPU, after which the renderer immediately uses these outputs to draw rays and signal-strength overlays. Ordering is enforced on the GPU to ensure the renderer reads only completed propagation results, without requiring CPU waits or data transfers back to the CPU. The simulator keeps the scene geometry in GPU memory as a triangle representation used for ray–surface intersection. During propagation, the GPU writes out ray segments for visualisation and accumulates field contributions into grid buffers for heatmap generation. The resulting received-power field is stored in GPU textures and rendered directly onto scene surfaces such as in the model’s walls, ceiling, and floor using tri-planar mapping (See Fig. 1c, d). In addition, a dedicated receiver buffer maintains the received-power estimate at the active receiver position for interactive querying.

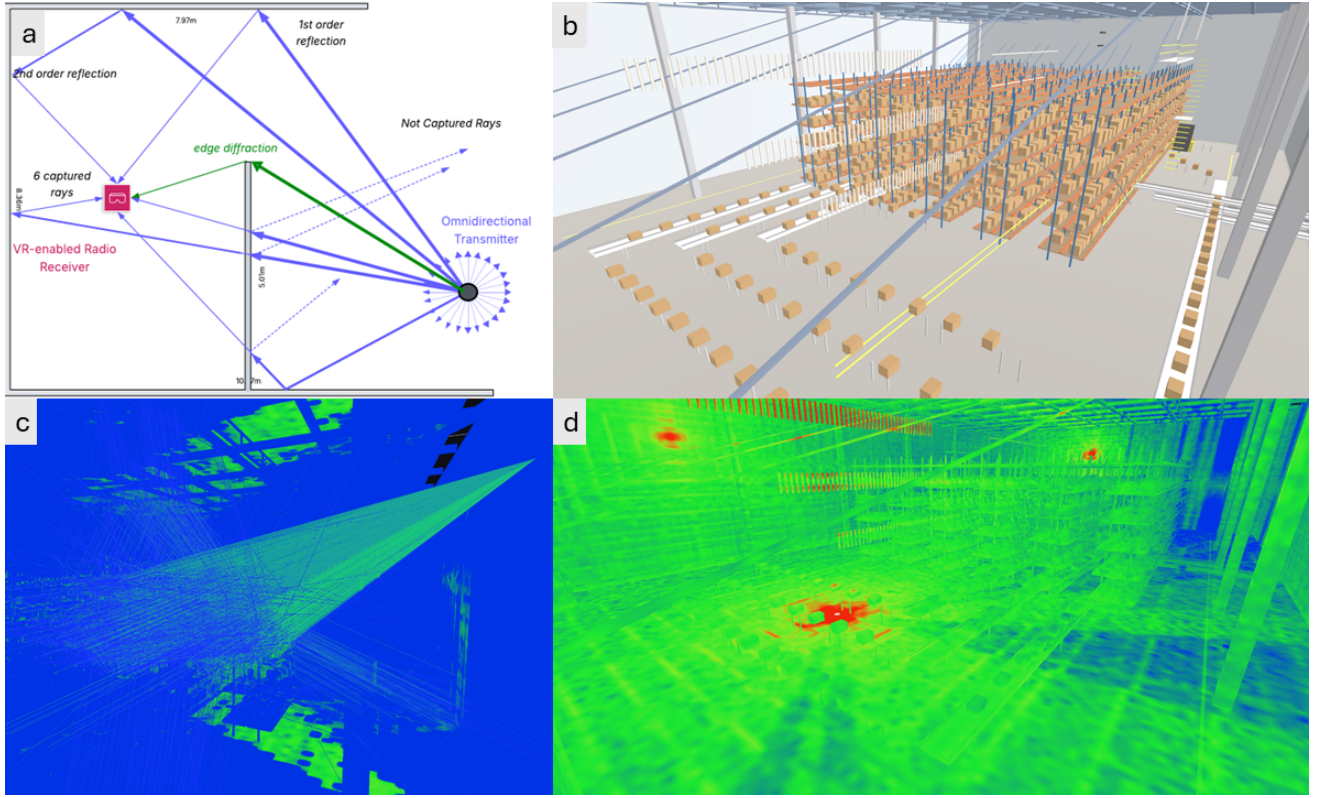


Fig. 1. GPU ray-tracing propagation visualisation examples showing multi-bounce rays and received-power heatmaps mapped onto room surfaces.

Dedicated compute-shader ray tracing performs propagation fully on the GPU, including multi-bounce traversal, polarisation handling, and spatial signal accumulation. Each GPU thread processes one ray-path slot and a fixed number of primary rays are generated around the transmitter, using either spherical (360°) or conical emission. Ray traversal proceeds for a specified number of reflections (currently 4), where for each ray the kernel identifies the closest hit against scene geometry and room bounds and updates the next interaction based on the surface normal. At each triangle hit, the kernel decomposes the complex electric-field vector into s and p orthogonal polarisation components in the plane of incidence, applies Fresnel coefficients $R_\perp$, $R_\parallel$, $T_\perp$, and $T_\parallel$, and reconstructs reflected and transmitted field vectors in the outgoing bases. The 3D model boundaries are treated as opaque reflectors to constrain the propagation domain. To maintain realistic estimation of the electric field performance, each ray keeps two active path slot; one for the horizontal and one for the parallel polarization and these are treated separately at each interaction. Their accumulated values are processed according to the polarization state of the receiving antenna. Along each traced path slots, the kernel samples points and accumulates complex field components (E_x, E_y, E_z) to facilitate the proper use of dyadic reflection, transmission and diffraction coefficients. A subsequent compute pass converts the accumulated field to received power (dBm) and writes the result into heatmap textures. Fig. 1 demonstrates examples of the visualisation of the

GPU-based radio-propagation simulation showing ray-tracing paths with received-power heatmaps mapped onto the scene surfaces. Fig. 1c shows individual rays and their reflections across the room, that can support line-of-sight, reflections, and occluded areas visualisation, demonstrating the ability for real-time, multi-perspective analysis of propagation behaviour and coverage patterns. Fig. 1d illustrates dense multi-bounce propagation simulations with the heatmap intensity indicating spatial variations in received signal strength and localised hotspots around high-interaction areas.

A. Prototype Functionalities

The prototype provides a number of interactive features to support radio propagation analysis through desktop mode or in VR (tethered VR) via OpenXR. To date, we have only tested the simulator using Meta Quest 2 VR headset. Both modes provide ray paths and heatmaps and navigating within the simulated environment. Prototype deployment of horizontal and vertical grids is under way for desktop version. The simulator supports loading arbitrary indoor scenes as glTF binary (GLB) models. After import, scene geometry is converted into a triangle representation for GPU intersection and used for ray traversal and heatmap projection on surfaces. Through the use of the PC keyboard in both modes, users can reposition the transmitter and receiver at runtime and recalculate the tracing output and the resulting received-power field. In VR mode, the user navigates using the wireless

controllers thumbsticks and simulation controls are mapped onto the controller's buttons.

The simulator supports i) single-ray stepping where rays can be traced and displayed one-by-one, supporting detailed inspection of individual multi-bounce paths; ii) ray-per-frame where a ray is generated and calculated after each calculated frame; and iii) all-rays mode where the full ray set can be generated, traced and visualised instantly. Users can toggle rays visualisation on and off at runtime to reduce visual clutter or isolate specific interaction types during analysis. Ray segments are colour-mapped according to their signal strength derived from received field/power along the path, providing an immediate visual indication of attenuation across distance and interactions helping to identify dominant paths versus negligible contributions. The signal strength colour map uses red to indicate very strong reception, yellow for moderately strong reception, green for moderate, and blue to indicate weaker reception. The received-power field is visualised as a heatmap applied across all relevant scene surfaces, including horizontal planes, walls and obstacles, supporting spatial interpretation of coverage within complex indoor geometry. For desktop mode, vertical and horizontal sampling planes capture the electric field across multiple levels using configurable cell-based grid, enabling to explore spatial variations in field intensity at different heights and cross-sections of the environment. Furthermore, users can point-and-select in the scene (camera picking currently in PC mode only) to query the received power in dBm at the picked point, providing a precise numerical reading.

B. Preliminary Performance Evaluation

To explore the simulator performance, a preliminary evaluation was performed across different scene complexities and ray counts to assess how geometric complexity and propagation workload affect frame rate performance in desktop and VR. For this experiment, a 3D warehouse setting with three scene complexity levels is used, comprising approximately 250k, 500k, and 750k triangles (Fig. 1b). For each scene, the simulator is executed with 30,000, 40,000, and 50,000 rays, and maintaining a fixed maximum interaction depth of four bounces. The test was conducted using an Intel I7 3.60GHz PC, with 32GB DDR4 RAM and equipped with a NVIDIA 4060 8GB GPU.

To ensure comparability, transmitter and receiver positions, propagation settings, and heatmap resolution were kept constant across all experiments. Each test condition was repeated three times, where user initiated a full ray simulation, disabled the generated rays to declutter the scene and navigated the environment through 4 different waypoints for 3 minutes. All configurations were executed in both desktop and tethered VR modes. The FPS threshold for VR is set at 60 FPS. The desktop version remained consistently above 90 FPS across all tested conditions and is therefore not included in Table I, which reports the VR results as the more demanding runtime condition. The table presents the average propagation time (ms) and average FPS across runs. Results indicate that the

lower-geometry scenes currently represent the most feasible deployment case for the prototype in VR, as they offer the best balance between propagation cost and interactive rendering performance. However, these results should be considered as initial baseline rather than the limit of the architecture, as there is substantial room for optimisation, particularly in reducing workload on geometry and ray traversal. For instance, more aggressive frustum and occlusion culling, spatial partitioning and improved acceleration-structure updates, can reduce the cost of propagation and rendering in higher geometry scenes. For PC, we have further experimented with even higher geometry counts (>1m) with consistently high frame rates.

TABLE I
VR PERFORMANCE EVALUATION RESULTS.

Vertices	Scene Triangles	Rays N	Avg Propagation Time (ms)	Avg FPS
263 153	250 000	30 000	81.73	72.8
		40 000	95.12	71.12
		50 000	112.05	70.26
383 444	500 000	30 000	115.4	61.11
		40 000	130.517	58.13
		50 000	165.798	55.97
516 224	750 000	30 000	118.573	39.821
		40 000	134.721	39.814
		50 000	158.921	39.77

To further examine the effect of scene geometric complexity on the received-power estimates, we compared captured values obtained from selected cells across different scene representations. To ensure comparability, the transmitter position and a fixed ray budget of 50,000 rays were maintained throughout the tests. We conducted multiple simulations in which the same randomly selected points on the horizontal and vertical sampling planes were measured across scenes with different geometric complexity to compare the received signal strength at each cell. For most sampled locations, the variation remained small, typically within about 1dB, while a few cells exhibited more noticeable differences in the higher-geometry case (between 1 to 4.5dB), with moderate variation in some cases and one larger deviation on the vertical plane. Overall, the results of the estimated received power were mostly consistent across the 200k, 500k, and 750k triangle scenes for most sampled cells, indicating that the dominant propagation behaviour is preserved despite the increase in mesh complexity. The results suggest that the simulator provides mostly consistent received-power estimates across varying scene complexities for most sampled locations, which is encouraging for the stability of the GPU-based implementation.

IV. DISCUSSION AND FUTURE WORK

A key contribution of this work is the development of a prototype engine that performs real time ray tracing and calculates radio propagation directly on GPU, enabling high-performance simulations for interactive analysis within an immersive environment. In a VR simulator in particular, usability is linked to frame stability and consistently high

refresh rates, where low frame rates and frame drops degrade interaction quality and increase discomfort and motion sickness [1]. Therefore, propagation simulation in VR has high performance and computation real-time requirements than conventional desktop visual analytics. Our previous experimentation with a CPU-based ray tracing prototype engine confirmed that the capabilities of CPU execution are limited, as ray counts, bounces, and scene complexity increases. The *VRadioSim* prototype presented in this paper addresses these constraints by exploiting the parallel nature of GPUs for propagation simulation and rendering by keeping propagation state and accumulation buffers on the GPU and minimises the CPU–GPU synchronisation and expensive readbacks during the simulation. This design supports significantly higher ray counts and richer multipath behaviour, as well as preserving high frame rate and stability required for interaction and visualisation. This enables users to experience low to medium complexity virtual spaces, reposition transmitter and receiver entities, toggle and step through ray paths, and query field values on surfaces without compromising the continuity of the desktop or the VR experience. Overall, the *VRadioSim* test results initially demonstrate that GPU-based propagation provides the capability for practical immersive radio simulation with responsive and physical detail usable in VR for propagation analysis.

A key limitation of the *VRadioSim* prototype is that while the propagation model and GPU pipeline have been designed to explore multipath signal behaviour and propagation, the system has not yet been validated for its accuracy against in situ measurements. As a result, the present work is regarded as exploratory and primarily intended to demonstrate feasibility, interactivity, and real-time visual inspection of propagation in immersive settings. The current performance results also indicate that the prototype is presently most suitable for lower-complexity indoor scenes particularly in tethered VR. Future work will prioritise systematic validation of the simulator through controlled in situ measurements in representative indoor environments, enabling quantitative comparison between predicted and observed received power. Further refinement and extension of the functionality will take place to capture, aggregate, and visualise ray interactions at multiple user-defined heights and cross-sections within the environment. Also optimisation of the GPU implementation to better support higher-complexity scenes in VR will take place together with inclusion of additional features such as spatial interaction UIs. A series of user experience evaluations of the VR component are planned to be conducted to assess usability, VR sickness, comfort, and simulator experience efficacy under realistic tasks, to determine how interactive visualisation influences user understanding during propagation analysis.

REFERENCES

- [1] L. Nisiotis, A. Anikina, and M. Raspopoulos, "The Development of a Virtual-Reality Wireless Signal Propagation Simulator in Unreal Engine: A Device and Performance Testing," *IT Professional*, vol. 28, no. 1, pp. 44–50, 2026.
- [2] L. Nisiotis, A. Anikina, and M. Raspopoulos, "Exploring gaming technologies, digital twins, and VR to visualise wireless propagation simulations," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 656–661, IEEE, 2024.
- [3] C.-X. Wang, J. Huang, H. Wang, X. Gao, X. You, and Y. Hao, "6G wireless channel measurements and models: Trends and challenges," *IEEE Vehicular Technology Magazine*, vol. 15, no. 4, pp. 22–32, 2020.
- [4] M. Born and E. Wolf, eds., *Principles of Optics - Electromagnetic Theory of Propagation, Interference and Diffraction of Light*. Pergamon, 6th edition ed., 1980. ISBN: 978-0-08-026482-0.
- [5] Z. Yun and M. F. Iskander, "Ray tracing for radio propagation modeling: Principles and applications," *IEEE Access*, vol. 3, pp. 1089–1100, 2015.
- [6] J. Tan, Z. Su, and Y. Long, "A full 3D GPU-based beam-tracing method for complex indoor environments propagation modeling," *IEEE Trans. Antennas and Propagation*, vol. 63, no. 6, pp. 2705–2718, 2015.
- [7] J. S. Lu, E. M. Vitucci, V. Degli-Esposti, F. Fuschini, M. Barbiroli, J. A. Blaha, and H. L. Bertoni, "A discrete environment-driven GPU-based ray launching algorithm," *IEEE Transactions on Antennas and Propagation*, vol. 67, no. 2, pp. 1180–1192, 2019.
- [8] Y. Liu, D. Shi, A. Li, and P. Zeng, "Radio wave propagation prediction based on the NVIDIA Optix GPU ray tracing engine," in *2019 IEEE 6th Intern. Symposium on Electromagnetic Compatibility*, pp. 1–3, 2019.
- [9] V. Degli-Esposti, F. Fuschini, E. M. Vitucci, M. Barbiroli, M. Zoli, L. Tian, X. Yin, D. A. Dupleich, R. Müller, C. Schneider, and R. S. Thomä, "Ray-tracing-based mm-wave beamforming assessment," *IEEE Access*, vol. 2, pp. 1314–1325, 2014.
- [10] J. Huang, C.-X. Wang, Y. Sun, J. Huang, and F.-C. Zheng, "A novel ray tracing based 6G RIS wireless channel model and RIS deployment studies in indoor scenarios," in *IEEE 33rd Intern. Symp. on Pers., Indoor and Mobile Radio Comms (PIMRC)*, pp. 884–889, 2022.
- [11] J. Huang, C.-X. Wang, Y. Sun, R. Feng, J. Huang, B. Guo, Z. Zhong, and T. J. Cui, "Reconfigurable intelligent surfaces: Channel characterization and modeling," *Proc. of the IEEE*, vol. 110, no. 9, pp. 1290–1311, 2022.
- [12] S. Hu, L.-X. Guo, Z. Liu, Z. Zhong, and Z. Nan, "A High-Performance GPU-Accelerated Ray-Tracing Method for Real-Time V2V Channel Modeling," *IEEE Antennas and Wireless Propagation Letters*, vol. 24, no. 8, pp. 2527–2531, 2025.
- [13] R. Kerris, "Ericsson builds digital twins for 5G networks in NVIDIA Omniverse," 2021. Online: <https://blogs.nvidia.com/blog/ericsson-digital-twins-omniverse/>.
- [14] Ericsson, "Next-generation simulation technology to accelerate the 5G journey," 2021. <https://www.ericsson.com/en/blog/2021/4/5g-simulation-omniverse-platform>.
- [15] D.-H. Tran, N. Waheed, Y. M. Saputra, et al., "Network Digital Twin for 6G and Beyond: An End-to-End View Across Multi-Domain Network Ecosystems," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 6866–6911, 2025.
- [16] L. Li, H. Sun, Z. Wang, W. Wang, and W. Fan, "Digital Twins of Electromagnetic Propagation Environments for Live 5G Networks—Part I: Channel Acquisition, EM Simulation, and Verification," *IEEE Transactions on Antennas and Propagation*, vol. 73, no. 4, 2025.
- [17] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Communications Surveys and Tutorials*, vol. 24, no. 4, pp. 2230–2254, 2022.
- [18] Z. Lai, D. Yuan, W. Bao, Y. Zhang, and B. B. Zhou, "DeepWiSim: A wireless signal simulator for automatic deep learning," in *Proceedings of the SIGCOMM '22 Poster and Demo Sessions*, p. 13–15, ACM, 2022.
- [19] P. Tang, J. Zhang, Y. Zhang, Y. Guan, P. Qi, F. Wang, L. Yu, and P. Zhang, "Demo abstract: "See the Radio Waves" via VR and its application in wireless communications," in *IEEE INFOCOM 2021 Workshops*, pp. 1–2, 2021.
- [20] J. Sköld, "Augmented visualisation of radio path propagation," 2021. Masters Dissertation, Umeå University, Department of Physics.
- [21] B. Levchenko, A. Chukhray, and D. Chumachenko, "Development of game modules with support for synchronous multiplayer based on Unreal Engine 4 using artificial intelligence approach," in *Integrated Computer Technologies in Mechanical Engineering*, pp. 503–513, 2020.
- [22] Z. Lai, C. Zhang, Y. Zhang, and D. Yuan, "DEMO: Real-Time Simulation of Wireless Signal Propagation for Dynamic Environment Through GPU-Based Ray Tracing," *ACM SIGCOMM*, p. 98–100, ACM, 2024.